

SKALA

2) AI의 서비스화데이터분석 및 AIOps

Python – IDE

2026.7

본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX

[참고] 2026년 현대 소프트웨어 개발 표준

스탠포드 대학의 CS146S: The Modern Software Developer 공개 커리큘럼 중 교육 연구 목적 내용 요약(by GPT)

2026년 개발 패러다임	• **SW 개발은 단순 구현이 아닌 AI를 활용한 설계 → 검증 → 조율 중심의 활동 전환**
AI Native 개발 Workflow	• **요구사항 정의부터 테스트 → 배포까지 전 단계에 AI가 개입** • **프롬프트 설계와 에이전트 활용 능력이 기본 역량**
현대 소프트웨어 개발자의 핵심 역량	• **언어 숙련도 외 시스템 설계 능력, 테스트 전략, 보안 인식, AI 결과 검증 능력 필수**
기술 스택 표준화 흐름	• **AI IDE, Agent 기반 개발 도구, Cloud Native 아키텍처, API-first 설계가 보편화**
조직 및 역할 변화	• **소규모 자율 팀 구조가 강화** • **시니어 개발자의 설계 →리뷰 → 의사결정 역할이 더욱 중요**
교육 및 평가 기준 변화	• **문제 풀이 중심 교육이 아닌 실제 시스템 구축 능력과 AI 활용 역량이 평가 기준**

AI – Native 개발 환경 구성



개발 환경 설정의 중요성

- 소프트웨어 개발에서 환경 설정은 단순한 도구 설치가 아님
- 환경구성 오류 시 팀 협업과 배포 과정에서 심각한 오류 발생함
- **2026년 현재 Python 생태계는 환경 관리 도구가 크게 발전하여, 과거 pip + venv 방식의 여러 한계를 극복한 현대적인 도구들이 표준으로 진행 중**

AI – Native 개발 환경 구성

uv
(차세대 Python 패키지 관리자)

- **Astral**이 개발한 **Rust** 기반의 초고속 **Python** 패키지 관리자
- **pip, virtualenv, poetry, pyenv**의 기능을 하나의 도구로 통합
- 기존 **pip** 대비 최대 **100**배 빠른 속도를 자랑

기존 방식 (pip + venv)	현대적 방식 (uv)
pip install requests	uv add requests
python -m venv .venv && source .venv/bin/activate	uv sync (자동 가상환경 생성)
requirements.txt 수동 관리	pyproject.toml + uv.lock 자동 관리
pip freeze > requirements.txt	uv lock (재현 가능한 잠금 파일)
느린 의존성 해결 (수십 초)	초고속 의존성 해결 (수 초 이하)
여러 도구 별도 설치 필요	uv 하나로 모든 것 해결

AI – Native 개발 환경 구성

uv 설치 및 프로젝트 초기화

```
# 1. uv 설치 (macOS/Linux)
curl -LsSf https://astral.sh/uv/install.sh | sh


# 2. 새 프로젝트 생성 (pyproject.toml 자동 생성)
uv init my-project
cd my-project


# 3. 의존성 추가
uv add pandas numpy fastapi uvicorn    # 프로덕션 의존성
uv add --dev pytest ruff mypy          # 개발 의존성
```

AI – Native 개발 환경 구성

uv 설치 및 프로젝트 초기화

- **# 4. 스크립트 실행**
- **uv run python main.py**

- **# 5. 환경 동기화 (다른 개발자가 프로젝트 받았을 때)**
- **uv sync**

- **# 6. 잠금 파일 생성**
- **uv lock**

AI – Native 개발 환경 구성

pyproject.toml 표준 프로젝트 구조

- pyproject.toml은 Python 프로젝트의 모든 설정을 하나의 파일에 담는 현대적인 표준
- 패키지 빌드 설정, 의존성, 도구 설정을 통합 관리

```
[project]
name = "my-ml-app"
version = "1.0.0"
description = "KDT MLOps 실습 프로젝트"
requires-python = ">=3.12"

dependencies = [
    "fastapi>=0.115",
    "pydantic>=2.0",
    "uvicorn[standard]",
    "pandas>=2.2",
    "polars>=1.0",
]
```

AI – Native 개발 환경 구성

pyproject.toml 표준 프로젝트 구조

- pyproject.toml은 Python 프로젝트의 모든 설정을 하나의 파일에 담는 현대적인 표준
- 패키지 빌드 설정, 의존성, 도구 설정을 통합 관리

```
[project.optional-dependencies]
dev = ["pytest>=8.0", "ruff>=0.8", "mypy>=1.13"]
ml  = ["torch>=2.5", "scikit-learn>=1.5"]

[tool.ruff]
line-length = 88
target-version = "py312"
select = ["E", "F", "I", "UP", "B", "SIM"]

[tool.mypy]
python_version = "3.12"
strict = true

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-ra -q --tb=short"
```

AI – Native 개발 환경 구성

AI 보조 코딩 도구 설정

2026년 소프트웨어 개발의 가장 큰 패러다임 전환은 AI 코드 어시스턴트의 일상화

도구	주요 기능	단축키/명령어	권장 사용법
Cursor IDE	AI 대화형 편집, @코드베이스 참조	Ctrl+K(편집), Ctrl+L(채팅)	전체 파일 맥락으로 코드 생성
Github Copilot	인라인 코드 제안	Tab(수락), Ctrl+Enter (후보)	반복 패턴 코드 자동 완성
/explain	선택 코드 설명 요청	채팅창에서 /explain 입력	처음 보는 코드에 대한 설명 요청
/fix	버그 자동 수정 제안	채팅창에서 /fix 입력	에러 메시지가 있을 때 활용
/tests	단위테스트 자동 생성	채팅창에서 /tests 입력	TDD 시작 단계에 활용

AI 도구는 코드 생성기가 아닌 pair programmer로 활용해주세요~
생성된 코드는 반드시 이해하고, 타입 검사와 테스트로 검증하셔야 합니다.

핵심 정리

uv = pip + venv + poetry를 하나로 통합한 차세대 패키지 관리자 (100배 빠름)

pyproject.toml은 현대 Python 프로젝트의 모든 설정을 담는 단일 파일 표준

AI 코드 어시스턴트는 생산성 향상 도구이며, 생성 코드 검증 책임은 개발자에게 있음

uv sync + uv lock으로 팀 전체가 동일한 환경을 재현 가능하게 구성